A파트(데이터 전처리 담당) 팀원이 작성한 문서 내용(25%/50% 기준 결측치 제거, Pass 데이터 중심의 6시그마 기반 클리핑, 중앙값 대치 등)과 현재 화면의 파이썬 코드를 연계하여 한 줄씩 쉽게 설명해 드리겠습니다.

1. 비전공자도 이해하기 쉬운 설명 (일상적 예시 포함)

① 데이터 불러오기 및 타겟 정리

Python

```
file_path = 'secom_25%_중앙값대치_이상치처리.xlsx'
df = pd.read_excel(file_path)
```

- **쉬운 설명**: A파트 친구가 이상한 값(튀는 값)을 다듬고 빈칸을 중앙값으로 채워둔 엑셀 파일(`secom_25%_중앙값대치_이상치처리.xlsx`)을 가져옵니다.

Python

```
y = df['Pass_Fail'].map({-1: 0, 1: 1})
X = df.drop(columns=['Result', 'Pass_Fail'])
```

- **쉬운 설명**: 컴퓨터는 보통 합격을 0, 불량을 1로 표시하는 걸 좋아합니다. 그래서 기존의 -1(합격)을 0으로, 1(불량)을 1로 바꾸어 정답지(`y`)를 만듭니다. 그리고 정답지와 상관없는 칼럼(`Result`, `Pass_Fail`)을 지운 나머지 센서 측정값 전체를 문제지(`X`)로 둡니다.

② 중요한 센서만 골라내는 함수 (`apply_feature_selection`)

개념 예시: 센서가 500개가 넘는데, 너무 비슷한 센서나 무의미한 센서가 많으면 시험 범위가 너무 넓어져 공부하기 힘듭니다. 공부하기 좋은 **알짜배기 30개 과목(센서)**만 골라내는 과정입니다.

Python

```
def apply_feature_selection(X_train, y_train, X_val, n_features=30):
```

- **쉬운 설명**: 훈련용 데이터에서 알짜배기 센서 30개를 선별해주는 '특집 선별기' 상자를 만듭니다.

Python

```
    corr_matrix = X_train.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    to_drop = [column for column in upper.columns if any(upper[column] > 0.95)]
    X_train_filter = X_train.drop(columns=to_drop)
    X_val_filter = X_val.drop(columns=to_drop)
```

- **쉬운 설명 (1단계)**: 쌍둥이처럼 서로 95% 이상 완전히 똑같이 움직이는 센서들을 찾습니다. 내용이 겹치면 하나만 남기고 나머지는 버립니다.

Python

```
    k_anova = min(100, X_train_filter.shape[1])
    selector_f = SelectKBest(score_func=f_classif, k=k_anova)
    selector_f.fit(X_train_filter, y_train)
    cols_anova = X_train_filter.columns[selector_f.get_support()]
    X_train_f = X_train_filter[cols_anova]
    X_val_f = X_val_filter[cols_anova]
```

- **쉬운 설명 (2단계)**: 불량품을 판별할 때 혼자서 유의미한 영향력을 행사하는 상위 100개 센서를 1차 후보로 꼽습니다.

Python

```
    rfe_estimator = LogisticRegression(class_weight='balanced', max_iter=1000, random
    k_rfe = min(n_features, X_train_f.shape[1])
    rfe = RFE(estimator=rfe_estimator, n_features_to_select=k_rfe, step=0.1)
    rfe.fit(X_train_f, y_train)
```

```
    selected_cols = X_train_f.columns[rfe.get_support()]
    return (X_train_f[selected_cols], X_val_f[selected_cols], list(selected_cols))
```

- **쉬운 설명 (3단계)**: 100개 후보 중에서 하나씩 떼어보며 테스트해보고, 가장 합작이 잘 되는 최정예 센서 30개만 최종 선택합니다.

③ AI 모델 준비 및 설정

Python

```
scale_pos_weight = (len(y) - sum(y)) / sum(y)
```

- **쉬운 설명**: 공정 데이터는 불량품(1)보다 정상품(0)이 훨씬 많습니다. 불량품 데이터 1개가 정상품 몇 개 분량의 가치가 있는지 비율을 계산합니다.

Python

```
models = {
    'kNN': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(class_weight='balanced', max_depth=5, ran
    'Logistic Regression': LogisticRegression(class_weight='balanced', max_iter=1000,
    'Random Forest': RandomForestClassifier(class_weight='balanced', n_estimators=100
    'XGBoost': XGBClassifier(scale_pos_weight=scale_pos_weight, eval_metric='logloss'
}
```

- **쉬운 설명**: 서로 다른 스타일을 가진 5명의 학생(알고리즘)을 모아 시험(불량 탐지)을 치르게 준비합니다.

④ 교차 검증 및 SMOTE 가공

Python

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
CUSTOM_THRESHOLD = 0.3
```

- **쉬운 설명**: 데이터를 5개 묶음으로 나누어 4개로 공부하고 1개로 시험을 치는 과정을 5번 반복(5-Fold)하도록 설정합니다. 그리고 "불량일 확률이 30%만 넘어도 일단 불량 의심품으로 판정해!"라며 기준을 엄격하게 잡습니다(미탐 방지).

Python

```python
for train_idx, val_idx in skf.split(X, y):
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]
```

- **쉬운 설명**: 5개 그룹 중 하나씩 돌아가며 학습 데이터와 검증(시험) 데이터를 나눕니다.

Python

```python
    imputer = SimpleImputer(strategy='median')
    scaler = StandardScaler()
    X_train_imp = pd.DataFrame(imputer.fit_transform(X_train), columns=X.columns)
    X_val_imp = pd.DataFrame(imputer.transform(X_val), columns=X.columns)
```

- **쉬운 설명**: 혹시 남아있는 빈칸은 중앙값으로 채웁니다(A파트의 전처리 규칙 준수).

Python

```python
    X_train_sel, X_val_sel, sel_cols = apply_feature_selection(X_train_imp, y_train,
    selected_features_list.extend(sel_cols)
```

- **쉬운 설명**: 앞서 만든 함수를 통해 최정예 센서 30개만 골라냅니다.

Python

```python
    X_train_proc = scaler.fit_transform(X_train_sel)
    X_val_proc = scaler.transform(X_val_sel)
```

- **쉬운 설명**: 센서마다 단위(cm, kg, °C 등)가 다르므로 평균 0, 표준편차 1로 키 맞추기(스케일링)를 해줍니다.

Python

```
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_proc, y_train)
```

- **쉬운 설명 (SMOTE)**: 불량품 데이터 수가 너무 적어서 AI가 제대로 배우지 못합니다. 불량품 데이터들의 특징을 흉내 내어 가짜 불량 데이터를 똑같이 복제해 늘려줍니다(오버샘플링).

Python

```
model.fit(X_train_res, y_train_res)
y_proba = model.predict_proba(X_val_proc)[:, 1]
y_pred_custom = (y_proba >= CUSTOM_THRESHOLD).astype(int)
```

- **쉬운 설명**: 모델에게 공부를 시키고, 검증용 데이터를 보여준 뒤 불량일 확률을 예측하게 합니다. 예측 확률이 30%(0.3) 이상이면 최종 불량(1)으로 판정합니다.

2. 전문 용어를 활용한 기술적 상세 설명

① 데이터 전처리 및 Target 변수 인코딩

Python

```
file_path = 'secom_25%_중앙값대치_이상치처리.xlsx'
df = pd.read_excel(file_path)
y = df['Pass_Fail'].map({-1: 0, 1: 1})
X = df.drop(columns=['Result', 'Pass_Fail'])
```

- **전문가 용어 설명**:
  - **Target Binarization (이진화)**: 범주형 레이블인 `-1(Pass)`과 `1(Fail)`을 머신러닝 손실 함수 계산에 적합한 `0`과 `1`로 매핑(Mapping)합니다.

- **Feature/Target Separation**: 메타데이터성 컬럼을 제거하고 독립 변수 Matrix(X)와 종속 변수 Vector(y)로 분리합니다.

② Feature Selection (특성 선택) 및 Data Leakage 방지

Python

```python
def apply_feature_selection(X_train, y_train, X_val, n_features=30):
    # 1. Multicollinearity Filtering
    corr_matrix = X_train.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    to_drop = [column for column in upper.columns if any(upper[column] > 0.95)]
    X_train_filter = X_train.drop(columns=to_drop)
    X_val_filter = X_val.drop(columns=to_drop)
```

- **전문가 용어 설명**:
  - **Multicollinearity (다공선성)**: 상관계수(r)가 0.95 이상인 고도로 연관된 피처를 제거하여 계수 추정의 분산을 줄이고 모델의 오버피팅을 방지합니다.
  - **Data Leakage (데이터 누수) 방지**: 피처 선택의 모든 통계량 계산(상관계수 등)을 오직 X_{train} 기준으로만 수행하여 검증 데이터 정보가 유출되는 것을 차단합니다.

Python

```python
    # 2. ANOVA F-test (Filter Method)
    k_anova = min(100, X_train_filter.shape[1])
    selector_f = SelectKBest(score_func=f_classif, k=k_anova)
    selector_f.fit(X_train_filter, y_train)
    cols_anova = X_train_filter.columns[selector_f.get_support()]
```

- **전문가 용어 설명**:
  - **Filter Method (ANOVA F-test)**: 각 연속형 피처와 범주형 Target 간의 집단 간 분산 대 집단 내 분산 비율(F-value)을 측정하여 일변량 통계적으로 유의미한 상위 100개 피처를 1차 선별합니다.

Python

```
    # 3. Recursive Feature Elimination (Wrapper Method)
    rfe_estimator = LogisticRegression(class_weight='balanced', max_iter=1000, random
    k_rfe = min(n_features, X_train_f.shape[1])
    rfe = RFE(estimator=rfe_estimator, n_features_to_select=k_rfe, step=0.1)
    rfe.fit(X_train_f, y_train)
    selected_cols = X_train_f.columns[rfe.get_support()]
    return (X_train_f[selected_cols], X_val_f[selected_cols], list(selected_cols))
```

- **전문가 용어 설명:**
  - **RFE (Wrapper Method):** 모델(Logistic Regression)을 반복적으로 학습시키며 가중치(Coefficient) 가 낮은 피처를 기하급수적으로 제거해 정제된 $n_features(=30)$개를 다변량 관점에서 도출합니다.

③ Stratified K-Fold & Data Preprocessing Pipeline

Python

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
CUSTOM_THRESHOLD = 0.3
```

- **전문가 용어 설명:**
  - **Stratified K-Fold:** 클래스 불균형(Class Imbalance) 문제를 고려하여 각 Fold 내 Pass/Fail 비율이 원본 데이터셋의 비율을 유지하도록 분할합니다.
  - **Threshold Tuning (임계값 조정):** 기본 디폴트 분류 threshold(0.5)를 0.3으로 하향 조율함으로써 FN(False Negative, 즉 불량을 정상으로 오분류) 오차 비용을 최소화하고 Recall(재현율)을 극대화합니다.

Python

```
    imputer = SimpleImputer(strategy='median')
    scaler = StandardScaler()
    X_train_imp = pd.DataFrame(imputer.fit_transform(X_train), columns=X.columns)
    X_val_imp = pd.DataFrame(imputer.transform(X_val), columns=X.columns)
    X_train_proc = scaler.fit_transform(X_train_sel)
    X_val_proc = scaler.transform(X_val_sel)
```

- **전문가 용어 설명**:
  - **Imputation (결측치 대치)**: 중앙값(Median) 대치를 적용하여 이상치(Outlier)에 강건(Robust)하게 결측값을 채웁니다.
  - **Standardization (표준화)**: $Z = \frac{X-\mu}{\sigma}$ 공식을 적용하여 각 피처의 평균을 0, 분산을 1로 정규화합니다.

Python

```
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_proc, y_train)
```

- **전문가 용어 설명**:
  - **SMOTE (Synthetic Minority Over-sampling Technique)**: 소수 클래스(Fail) 데이터의 k-최근접 이웃(k-NN) 간의 직선상에 위치하는 가상의 합성 샘플을 생성하여 클래스 불균형을 해소합니다. (검증 세트에는 적용하지 않아 평가의 객관성을 보장합니다.)

Python

```
model.fit(X_train_res, y_train_res)
y_proba = model.predict_proba(X_val_proc)[:, 1]
y_pred_custom = (y_proba >= CUSTOM_THRESHOLD).astype(int)
```

- **전문가 용어 설명**:
  - **Probability Prediction & Classification**: 모델에서 산출된 양성 클래스(Fail) 추정 확률값 $P(Y=1\vert{}X)$을 사용자 정의 임계값 `CUSTOM_THRESHOLD`와 비교하여 최종 예측 레이블을 0 또는 1로 결정합니다.